

LAPORAN PARKTIKUM 2

SISTEM OPERASI



Dosen Pengampu Mata Kuliah:

Triawan Adi Cahyanto, M.Kom

Disusun Oleh:

Mohamad Qowi Aizani (2410651003)

PRODI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

TUJUAN

1. Memahami konsep proses dan siklus hidupnya.
2. Mampu membuat dan mengelola proses menggunakan system call seperti `fork()`, `wait()`, dan `exec()`.
3. Memahami hubungan antara parent process dan child process.
4. Menganalisis Process ID (PID) dan Process Control Block (PCB).
5. Mempelajari komunikasi antar proses (Interprocess Communication / IPC) menggunakan *pipe* dan *shared memory*.
6. Membandingkan performa metode IPC

Dasar Teori

A. Konsep Proses

Proses adalah program yang sedang dieksekusi oleh sistem operasi. Setiap proses memiliki identitas unik yang disebut Process ID (PID). Dalam sistem operasi seperti Linux, proses dapat membuat proses lain menggunakan system call `fork()`. Proses yang membuat disebut *parent*, sedangkan proses hasil duplikasi disebut *child process*.

B. Siklus Hidup Proses

Proses mengalami beberapa status selama hidupnya, yaitu *new*, *ready*, *running*, *waiting*, dan *terminated*. Sistem operasi mengatur transisi antar status melalui scheduler.

C. System Call `fork()` dan `wait()`

- `fork()` digunakan untuk membuat proses baru (child).
- `wait()` digunakan untuk menunggu proses baru (child) selesai.

D. Interprocess Communication (IPC)

IPC adalah mekanisme komunikasi antar proses. Metode yang umum digunakan antara lain:

- Pipes – komunikasi satu arah antar proses parent-child.
- Named Pipes (FIFO) – versi pipe dengan nama yang bisa diakses banyak proses.
- Shared Memory – proses berbagi area memori yang sama untuk bertukar data, umumnya lebih cepat dari pipe.

PERCOBAAN

1. Percobaan 1.1

Membuat proses sederhana kode program (process_basic.c):

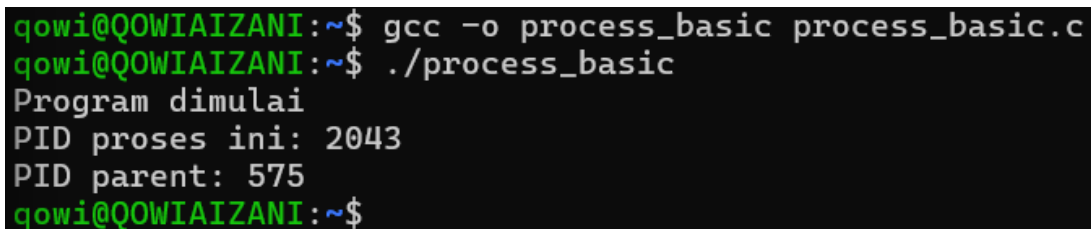
```
#include <stdio.h>
#include
<unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid;

    printf("Program dimulai\n");
    printf("PID proses ini: %d\n", getpid());
    printf("PID parent: %d\n", getppid());

    return 0;
}
```

Output:



```
qowi@QOWIAIZANI:~$ gcc -o process_basic process_basic.c
qowi@QOWIAIZANI:~$ ./process_basic
Program dimulai
PID proses ini: 2043
PID parent: 575
qowi@QOWIAIZANI:~$
```

Analisis:

- Baris “**Program dimulai**” dicetak saat program mulai dijalankan.
- Baris “**PID proses ini: 2043**” menampilkan **Process ID (PID)** dari proses program kamu sendiri.
 - Setiap program yang berjalan di sistem operasi (Linux) mendapat nomor unik bernama **PID**.
 - Dalam kasus ini, program kamu memiliki PID **2043**.
- Baris “**PID parent: 575**” menunjukkan **Parent Process ID (PPID)**, yaitu **PID** dari proses yang menjalankan program ini.
 - Biasanya, program kamu dijalankan oleh **shell** (misalnya bash atau zsh), dan PID shell itu adalah **575**.

2. Percobaan 1.2

Parent dan Child Process (fork_demo.c)

```
#include
<stdio.h>
#include
<unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid;

    printf("Sebelum fork()\n");
    printf("PID: %d\n\n",

    getpid()); pid = fork();

    if (pid < 0) {
        fprintf(stderr, "Fork gagal!\n");
        return 1;
    }
    else if (pid == 0) {
        // Child process
        printf("CHILD PROCESS\n");
        printf("PID saya: %d\n", getpid());
        printf("PID parent saya: %d\n", getppid());
    }
    else {
        // Parent process
        printf("PARENT PROCESS\n");
        printf("PID saya: %d\n",
        getpid()); printf("PID child
        saya: %d\n", pid);
    }

    printf("Proses %d selesai\n\n", getpid());

    return 0;
}
```

Output:

```
qowi@QOWIAIZANI:~$ gcc -o fork_demo fork_demo.c
qowi@QOWIAIZANI:~$ ./fork_demo
Sebelum fork()
PID: 2080

PARENT PROCESS
PID saya: 2080
PID child saya: 2081
Proses 2080 selesai

CHILD PROCESS
PID saya: 2081
PID parent saya: 570
Proses 2081 selesai
```

Analisis:

Bagian Output dan Penjelasan

- "Sebelum fork()" : Dicitak sebelum proses bercabang. Artinya, hanya ada satu proses saat ini (PID 2080).
- fork() dipanggil : Membuat salinan proses baru disebut child process. Sekarang ada dua proses: parent dan child.
- Parent process (PID 2080) : Parent tetap memiliki PID yang sama (2080). Nilai pid dari fork() berisi PID anak (2081).
- Child process (PID 2081) : Child memiliki PID baru (2081). Nilai pid di child = 0, jadi masuk ke blok if (pid=0).
- PID parent saya: 570 : Menunjukkan bahwa parent proses (2080) dijalankan oleh proses shell (PID 570). Jadi child melihat PPID = 570 karena parent (2080) sudah selesai lebih dulu, dan PPID otomatis dialihkan ke init/shell.
- "Proses... selesai" : Masing-masing proses (parent dan child) menampilkan bahwa eksekusinya telah berakhir.

3. Percobaan 1.3

Process termination dengan wait() kode program (process_wait.c)

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid;
    int status;

    pid = fork(); if
    (pid == 0) {
        // Child process
        printf("Child: PID saya %d, akan tidur 3 detik\n", getpid()); sleep(3);
        printf("Child: Selesai!\n"); return
        42; // Exit code
    }
    else {
        // Parent process
        printf("Parent: Menunggu child (PID: %d) selesai...\n", pid); wait(&status);

        if (WIFEXITED(status)) {
            printf("Parent: Child selesai dengan exit code: %d\n", WEXITSTATUS(status));
        }
    }

    return 0;
}
```

Output:

```
qowi@QOWIAIZANI:~$ gcc -o process_wait process_wait.c
qowi@QOWIAIZANI:~$ ./process_wait
Parent: Menunggu child (PID: 2115) selesai...
Child: PID saya 2115, akan tidur 3 detik
Child: Selesai!
Parent: Child selesai dengan exit code: 42
qowi@QOWIAIZANI:~$ |
```

Analisis:

- Parent: Menunggu child (PID: 2115) selesai...: Proses induk (parent) membuat proses anak dengan fork(). Parent lalu memanggil wait() untuk menunggu child selesai.
- Child: PID saya 2115, akan tidur 3 detik : Proses anak menjalankan blok if (pid == 0) dan menampilkan PID-nya sendiri, lalu tidur 3 detik menggunakan sleep(3).
- Child: Selesai! : Setelah tidur 3 detik, child keluar dari program dengan exit(42).
- Parent: Child selesai dengan exit code: 42 : Setelah child selesai, parent menerima sinyal dari wait() bahwa child telah berakhir, dan membaca kode keluar (exit code) yang dikembalikan anak, yaitu 42.

4. Percobaan 1.4

Multiple child processes kode program (multiple_fork.c)

```
#include
<stdio.h> #include
<unistd.h>
#include <sys/wait.h>

int main() {
    int i;
    pid_t pid;
    int num_children = 3;

    printf("Parent PID: %d\n\n", getpid());

    for (i = 0; i < num_children; i++) {
        pid = fork();

        if (pid == 0) {
            // Child process
            printf("Child %d: PID = %d, Parent PID = %d\n", i+1, getpid(), getppid());
            sleep(i + 1); // Simulasi pekerjaan printf("Child %d selesai\n", i+1);
            return i;
        }
    }
}
```

```

    }
}

// Parent menunggu semua child
for (i = 0; i < num_children; i++) { int
    status;
    pid_t child_pid = wait(&status);
    printf("Parent: Child dengan PID %d selesai\n", child_pid);
}

printf("\nParent: Semua child selesai\n"); return

0;
}

```

Output:

```

qowi@QOWIAIZANI:~$ gcc -o multiple_fork multiple_fork.c
qowi@QOWIAIZANI:~$ ./multiple_fork
Parent PID: 2150

Child 1: PID = 2151, Parent PID = 2150
Child 2: PID = 2152, Parent PID = 2150
Child 3: PID = 2153, Parent PID = 2150
Child 1 selesai
Parent: Child dengan PID 2151 selesai
Child 2 selesai
Parent: Child dengan PID 2152 selesai
Child 3 selesai
Parent: Child dengan PID 2153 selesai

Parent: Semua child selesai
qowi@QOWIAIZANI:~$ |

```

Analisi:

- Parent PID: 2150 : Proses utama (parent) memiliki ID 2150. Ini adalah proses yang akan membuat 3 anak.
- Child 1-3 dibuat (PID 2151-2153) : Loop for memanggil fork() tiga kali, sehingga terbentuk 3 proses anak. Setelah anak punya PID berbeda, tapi Parent PID 2150.
- Child 1 selesai, Child 2 selesai, Child 3 selesai : Setiap child mencetak pesan sebelum keluar (return 0/exit(0)), menandakan tugas mereka selesai.
- Parent: Child dengan PID 2151-2153 selesai : Parent memanggil wait() sebanyak 3 kali - setiap pemanggilan menunggu satu child selesai, dan menampilkan PID-nya.
- Parent: Semua child selesai : Setelah semua proses anak berakhir, parent juga

mengakhiri eksekusi.

1. Percobaan 2.1

Pipes (One Way Communication) kode program (pipe_demo.c)

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    int pipefd[2]; // pipefd[0] = read, pipefd[1] = write
    pid_t pid;
    char write_msg[] = "Hello dari Parent!";
    char read_msg[100];

    // Buat pipe
    if (pipe(pipefd) == -1) {
        perror("Pipe gagal");
        return 1;
    }

    pid = fork();

    if (pid < 0) {
        perror("Fork gagal");
        return 1;
    }
    else if (pid == 0) {
        // Child process - membaca dari pipe
        close(pipefd[1]); // Tutup ujung write
        read(pipefd[0], read_msg, sizeof(read_msg));
```

```

        printf("Child menerima: %s\n", read_msg);
        close(pipefd[0]);
    }
    else {
        // Parent process - menulis ke pipe
        close(pipefd[0]); // Tutup ujung read
        printf("Parent mengirim: %s\n", write_msg);
        write(pipefd[1], write_msg, strlen(write_msg) + 1);
        close(pipefd[1]);
        wait(NULL); // Tunggu child selesai
    }

    return 0;
}

```

Output:

```

qowi@QOWIAIZANI:~$ gcc -o pipe_demo pipe_demo.c
qowi@QOWIAIZANI:~$ ./pipe_demo
Parent mengirim: Hello dari Parent!
Child menerima: Hello dari Parent!
qowi@QOWIAIZANI:~$ |

```

Analisis:

1. pipe(fd) : Membuat sebuah saluran komunikasi satu arah antara proses parent dan child. fd[0] digunakan untuk membaca, fd[1] digunakan untuk menulis.
2. fork() : Membuat child process. Setelah fork(), ada dua proses yang berjalan parent dan child. Keduanya memiliki salinan fd.
3. Parent Process
 - Menutup ujung baca (close(fd[0])) karena hanya ingin mengirim data.
 - Mengirim string "Hello dari Parent!" ke fd[1] dengan write()
 - Menampilkan pesan "Parent mengirim: ...".
4. Child Process
 - Menutup ujung tulis (close(fd[1])) karena hanya ingin menerima data.
 - Membaca data dari fd[0] menggunakan read().

- Menampilkan hasil "Child menerima: Hello dari Parent!".

5. Sinkronisasi alami : Karena pipe bersifat blocking, proses child akan menunggu sampai ada data yang dikirim oleh parent. Sehingga urutan output selalu konsisten.

2. Percobaan 2.2

Named Pipes (FIFO) program: fifo_writer.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>

#define FIFO_FILE

"/tmp/my_fifo" int main() {
    int fd;
    char buffer[100];

    // Buat FIFO jika belum ada
    mkfifo(FIFO_FILE, 0666);

    printf("Writer: Menunggu reader...\n"); fd =
    open(FIFO_FILE, O_WRONLY);

    printf("Masukkan pesan: ");
    fgets(buffer, sizeof(buffer), stdin);

    write(fd, buffer, strlen(buffer) + 1);
    printf("Writer: Pesan terkirim\n");

    close(fd);

    return 0;
}
```

dan fifo_reader.c

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>
```

```

#define FIFO_FILE

"/tmp/my_fifo" int main() {
    int fd;
    char buffer[100];

    printf("Reader: Membuka FIFO...\n"); fd
    = open(FIFO_FILE, O_RDONLY);

    read(fd, buffer, sizeof(buffer));
    printf("Reader: Menerima pesan: %s\n", buffer);

    close(fd);
    unlink(FIFO_FILE);    // Hapus FIFO

    return 0;
}

```

Output:

```

qowi@QOWIAIZANI:~$ gcc -o fifo_writer fifo_writer.c
qowi@QOWIAIZANI:~$ ./fifo_writer
Writer: Menunggu reader...
Masukkan pesan: hello world
Writer: Pesan terkirim
qowi@QOWIAIZANI:~$ |

```

```

qowi@QOWIAIZANI:~$ gcc -o fifo_reader fifo_reader.c
qowi@QOWIAIZANI:~$ ./fifo_reader
Reader: Membuka FIFO...
Reader: Menerima pesan: hello world

qowi@QOWIAIZANI:~$ |

```

Analisis:

- Writer: Menunggu reader...: Writer belum bisa menulis karena belum ada proses lain yang membuka FIFO untuk membaca.
- Reader: Membuka FIFO...: Saat reader dijalankan, writer baru dapat lanjut mengirim data.
- Masukkan pesan: hello world : User memasukkan teks ke writer.
- Writer: Pesan terkirim : Pesan berhasil dikirim ke FIFO.
- Reader: Menerima pesan: hello world : Reader berhasil membaca data yang dikirim

writer melalui FIFO.

3. Percobaan 2.3

Shared Memory IPC kode program shm_writer.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>

#define SHM_SIZE

1024 int main() {
    key_t key;
    int shmid;
    char*shared_memory;
    char message[100];

    // Generate unique key
    key = ftok("shmfile", 65);

    // Create shared memory
    shmid = shmget(key, SHM_SIZE, 0666 | IPC_CREAT);

    // Attach to shared memory
    shared_memory = (char*) shmat(shmid, NULL, 0);

    printf("Writer: Masukkan pesan: ");
    fgets(message, sizeof(message), stdin);

    // Write to shared memory
    strcpy(shared_memory, message);
    printf("Writer: Data ditulis ke shared memory\n");

    // Detach from shared memory
    shmdt(shared_memory);

    return 0;
}
```

dan shm_reader.c

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/shm.h>
```

```

#define SHM_SIZE

1024 int main() {
    key_t key;
    int shmid;
    char *shared_memory;

    // Generate same key
    key = ftok("shmfile", 65);

    // Access shared memory
    shmid = shmget(key, SHM_SIZE, 0666);

    // Attach to shared memory
    shared_memory = (char*) shmat(shmid, NULL, 0);

    // Read from shared memory
    printf("Reader: Data dari shared memory: %s\n", shared_memory);

    // Detach and remove shared memory
    shmdt(shared_memory);
    shmctl(shmid, IPC_RMID, NULL);

    return 0;
}

```

Output:

```

qowi@QOWIAIZANI:~$ touch shmfile
qowi@QOWIAIZANI:~$ gcc -o shm_writer shm_writer.c
qowi@QOWIAIZANI:~$ ./shm_writer
Writer: Masukkan pesan: hello world
Writer: Data ditulis ke shared memory
qowi@QOWIAIZANI:~$ |

qowi@QOWIAIZANI:~$ touch shmfile
qowi@QOWIAIZANI:~$ gcc -o shm_reader shm_reader.c
qowi@QOWIAIZANI:~$ ./shm_reader
Reader: Data dari shared memory: hello world

qowi@QOWIAIZANI:~$

```

Analisis:

Program menunjukkan komunikasi antar proses menggunakan **Shared Memory (memori bersama)**.

- **shm_writer** menulis pesan "hello world" ke shared memory.
- **shm_reader** membaca pesan yang sama dari shared memory.

Kedua proses menggunakan **key yang sama** (ftok("shmfile", 65)) sehingga terhubung ke area memori yang sama.

shmget() membuat/akses memori, shmat() menempelkan memori ke proses, dan shmdt() melepaskannya setelah selesai.

4. Percobaan 2.4

Perbandingan performa IPC kode (ipc_benchmark.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/wait.h>
#include <sys/time.h>

#define DATA_SIZE 100
#define SHM_SIZE (DATA_SIZE * sizeof(int))

double get_time() {
    struct timeval t;
    gettimeofday(&t, NULL);
    return t.tv_sec + t.tv_usec / 1000000.0;
}

void test_shared_memory() {
    key_t key = 1234;
```

```

int shmid;
int *shared_data;
pid_t pid;
double start, end;

// Buat shared memory
shmid = shmget(key, SHM_SIZE, 0666 | IPC_CREAT);
if (shmid < 0) {
    perror("shmget gagal");
    exit(1);
}

start = get_time();
pid = fork();

if (pid < 0) {
    perror("fork gagal");
    exit(1);
}
else if (pid == 0) {
    // Child process - membaca data
    shared_data = (int *) shmat(shmid, NULL, 0);
    if (shared_data == (void *) -1) {
        perror("shmat gagal di child");
        exit(1);
    }

    int sum = 0;
    for (int i = 0; i < DATA_SIZE; i++) {
        sum += shared_data[i];
    }
}

```



```

        shmdt(shared_data);
        exit(0);
    }
    else {
        // Parent process - menulis data
        shared_data = (int *) shmat(shmid, NULL, 0);
        if (shared_data == (void *) -1) {
            perror("shmat gagal di parent");
            exit(1);
        }

        for (int i = 0; i < DATA_SIZE; i++) {
            shared_data[i] = i;
        }

        wait(NULL);
        end = get_time();

        printf("Shared Memory: %.6f detik\n", end - start);

        shmdt(shared_data);
        shmctl(shmid, IPC_RMID, NULL); // Hapus shared memory
    }
}

void test_pipe() {
    int pipefd[2];
    pid_t pid;
    double start, end;
    int data[DATA_SIZE];

```

```

int buffer[DATA_SIZE];

if (pipe(pipefd) == -1) {
    perror("pipe gagal");
    exit(1);
}

start = get_time();
pid = fork();

if (pid < 0) {
    perror("fork gagal");
    exit(1);
}
else if (pid == 0) {
    // Child membaca dari pipe
    close(pipefd[1]);
    read(pipefd[0], buffer, sizeof(buffer));
    close(pipefd[0]);
    exit(0);
}
else {
    // Parent menulis ke pipe
    close(pipefd[0]);
    for (int i = 0; i < DATA_SIZE; i++) {
        data[i] = i;
    }
    write(pipefd[1], data, sizeof(data));
    close(pipefd[1]);
    wait(NULL);
    end = get_time();
}

```

```

        printf("Pipe: %.6f detik\n", end - start);
    }
}

int main() {
    printf("Benchmarking IPC Methods dengan %d integers\n\n", DATA_SIZE);

    test_pipe();
    test_shared_memory();

    return 0;
}

```

Output:

```

qowi@QOWIAIZANI:~$ gcc -o ipc_benchmark ipc_benchmark.c
qowi@QOWIAIZANI:~$ ./ipc_benchmark
Benchmarking IPC Methods dengan 100 integers

Pipe: 0.000853 detik
Shared Memory: 0.000579 detik
qowi@QOWIAIZANI:~$ |

```

Analisis:

Shared Memory lebih cepat (0.000579 detik) dibanding Pipe (0.000853 detik) karena akses langsung ke memori bersama tanpa lewat kernel, sedangkan Pipe memerlukan proses I/O tambahan.

KESIMPULAN

1. Proses adalah satuan kerja dasar dalam sistem operasi yang dapat membuat proses lain menggunakan `fork()`.
2. Hubungan parent-child dapat diamati melalui PID dan `wait()` yang digunakan untuk sinkronisasi.
3. Interprocess Communication (IPC) penting untuk pertukaran data antar proses.
4. *Pipes* cocok untuk komunikasi sederhana, sementara *shared memory* lebih efisien untuk volume data besar.
5. Melalui percobaan ini, mahasiswa memahami mekanisme dasar pengelolaan proses dan komunikasi antar proses di sistem operasi Linux.

